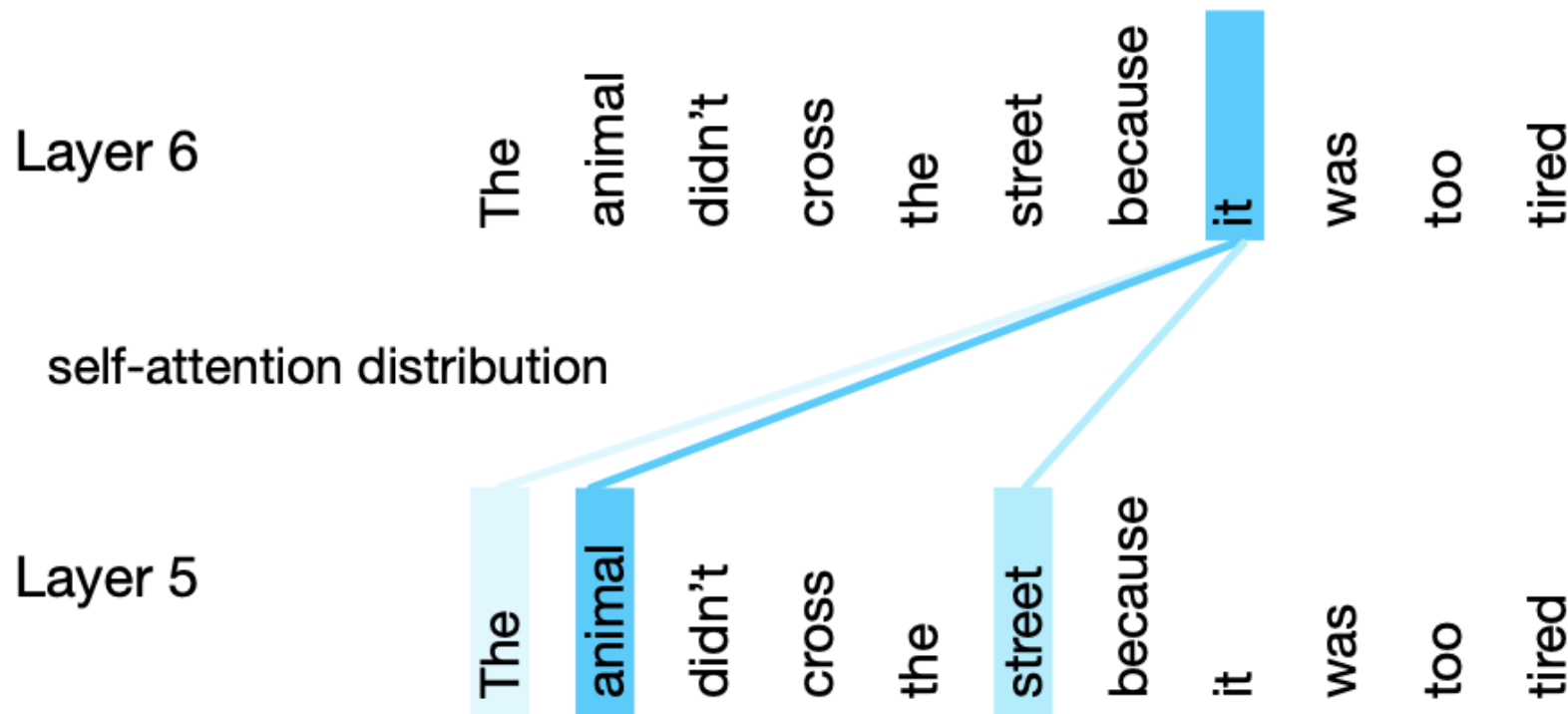


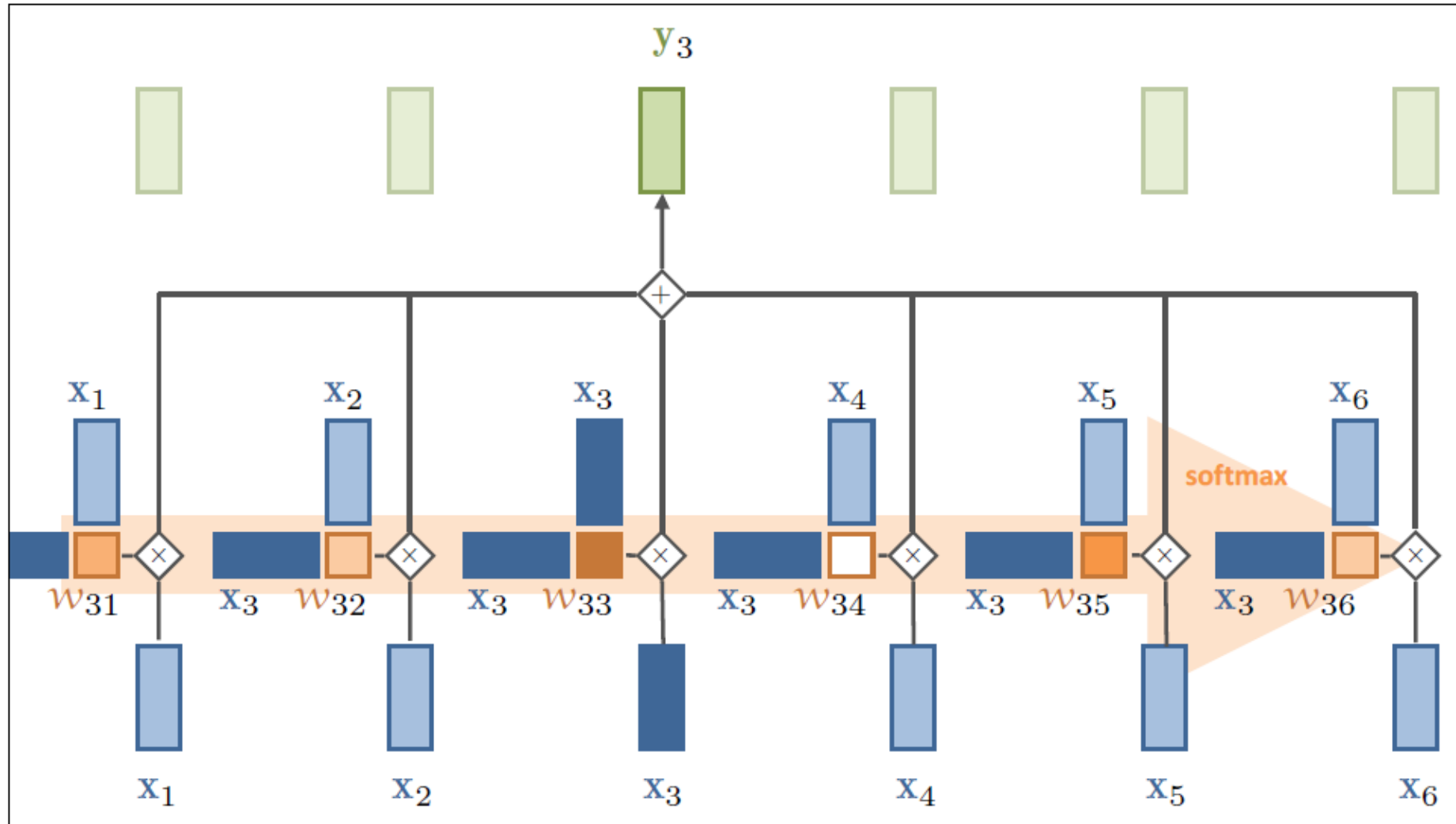
More on Attention

Self Attention

Self Attention



SA: Simple Self Attention



$$y_i = \sum_j w_{ij} x_j$$

$$w_{ij}^0 = x_i^T x_j$$

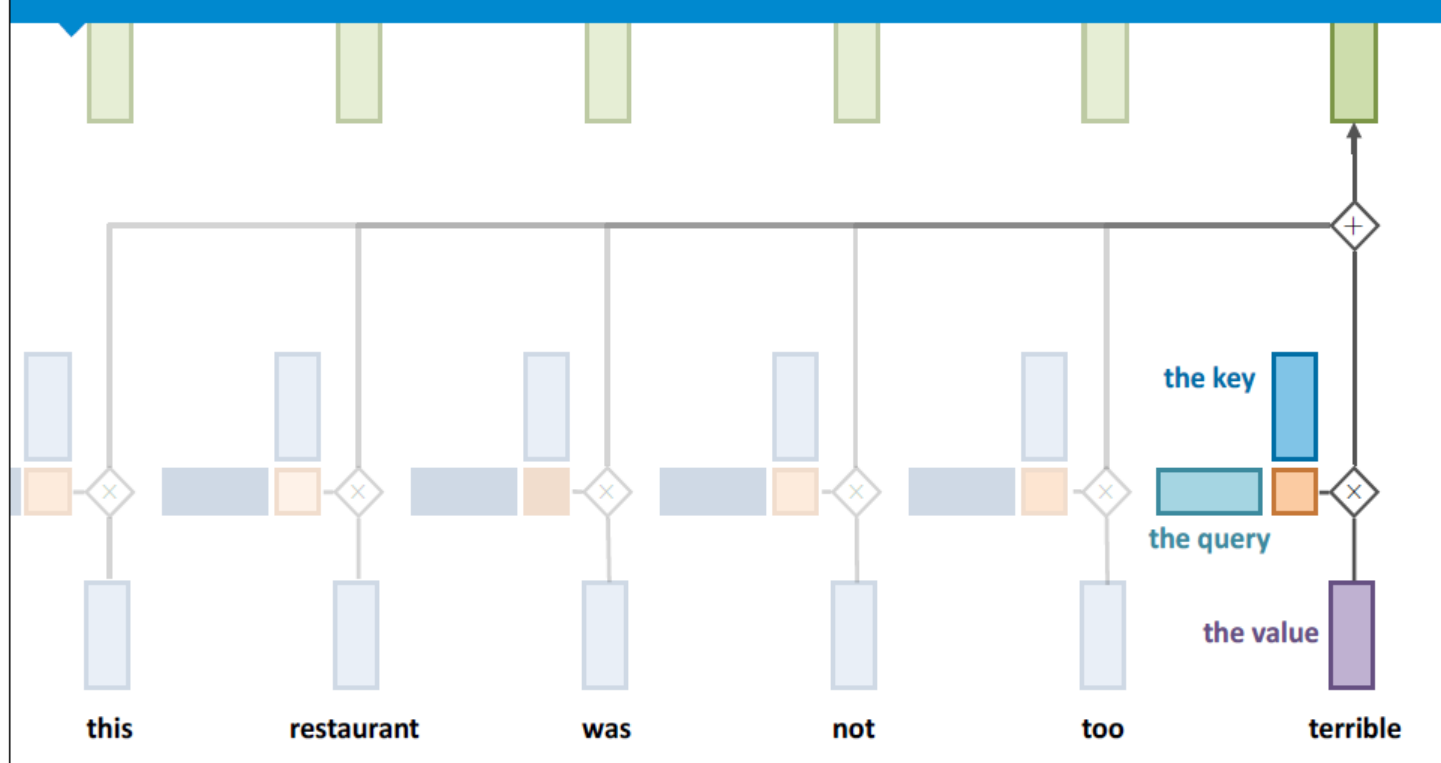
$$w_{ij} = \frac{\exp w_{ij}^0}{\sum_l \exp w_{il}^0}$$

SA: Observations

- Input and Output:
 - Vectors are of same in number and dimension
 - Set of vectors (in simple form). Add position, if required.
- Every output vector is influenced by “all” input vectors.
- “W” is not learned here. It is computed from the input/data.

Query, Key and Value

KEYS, QUERIES AND VALUES



In each self attention computation, every input vector occurs in three distinct roles:

- **the value**: the vector that is used in the weighted sum that ultimately provides the output
- **the query**: the input vector that corresponds to the current output, matched against every other input vector.
- **the key**: the input vector that the query is matched against to determine the weight.

Query, Key and Value

- As *the current focus of attention* when being compared to all of the other preceding inputs. We'll refer to this role as a **query**.
- In its role as *a preceding input* being compared to the current focus of attention. We'll refer to this role as a **key**.
- And finally, as a **value** used to compute the output for the current focus of attention.

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

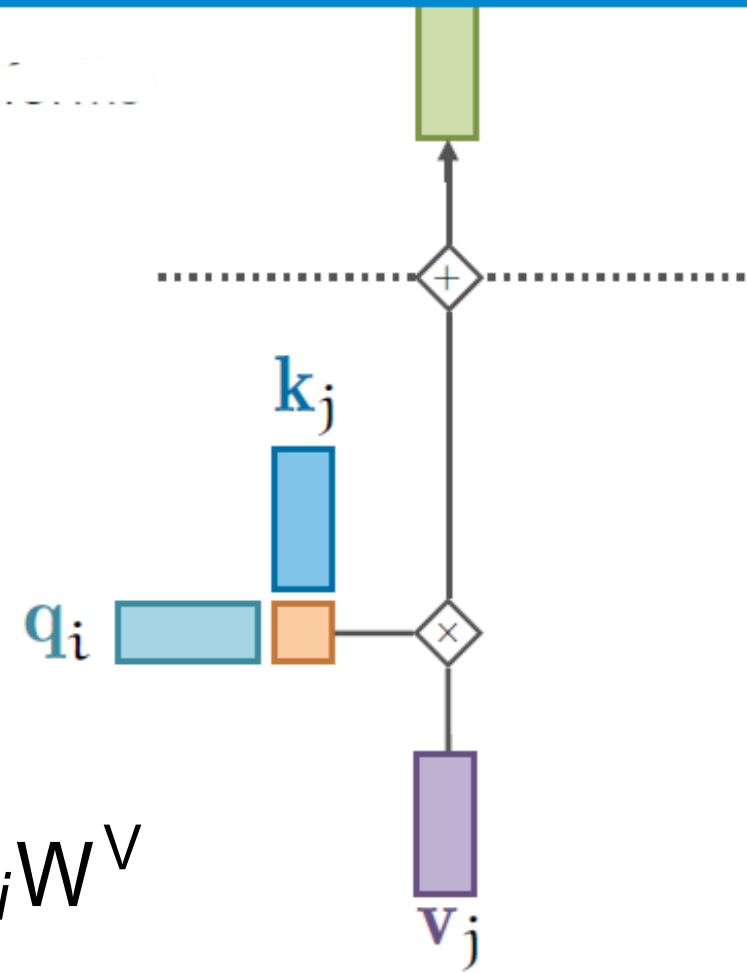
Learnable “Query”, “Key” and “Value”

$$\mathbf{k}_i = \mathbf{K}\mathbf{x}_i + \mathbf{b}_k$$

$$\mathbf{q}_i = \mathbf{Q}\mathbf{x}_i + \mathbf{b}_q$$

$$\mathbf{v}_i = \mathbf{V}\mathbf{x}_i + \mathbf{b}_v$$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$



Blank Slide

Matrix View

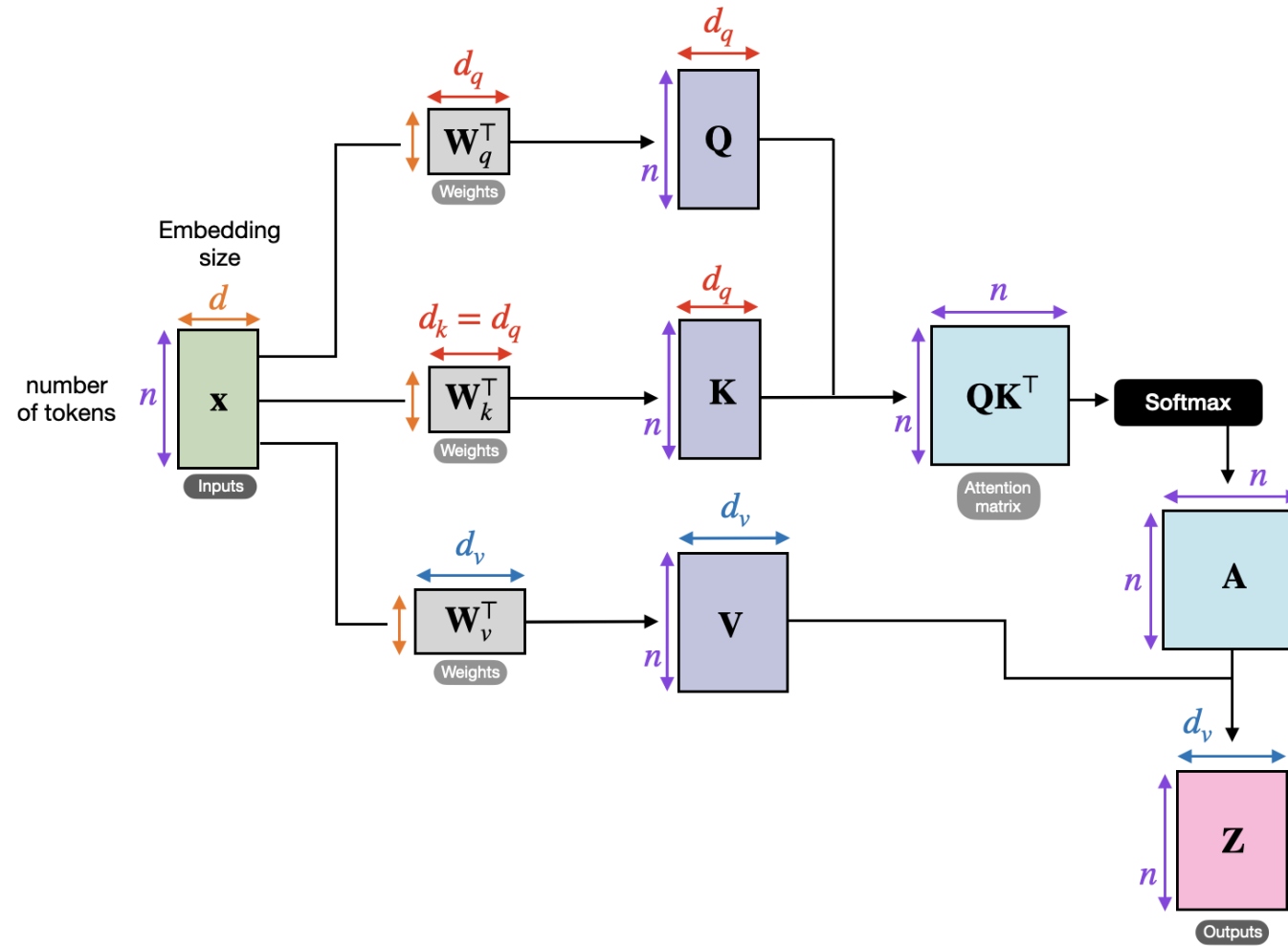
$$\begin{array}{c} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{array} \times \begin{array}{c} \text{W}^Q \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array} = \begin{array}{c} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array}$$

$$\begin{array}{c} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{array} \times \begin{array}{c} \text{W}^K \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array} = \begin{array}{c} \text{K} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array}$$

$$\begin{array}{c} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{array} \times \begin{array}{c} \text{W}^V \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array} = \begin{array}{c} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array}$$

$$\begin{aligned}
 & \text{softmax} \left(\frac{\begin{array}{c} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array} \times \begin{array}{c} \text{K}^T \\ \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} \end{array}}{\sqrt{d_k}} \right) \begin{array}{c} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array} \\
 & = \begin{array}{c} \text{Z} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array}
 \end{aligned}$$

Self Attention: Computation



Blank Slide

Parallelizable Computation

$$q_i = x_i W^Q; k_i = x_i W^K; v_i = x_i W^V$$

$$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{d_k}$$

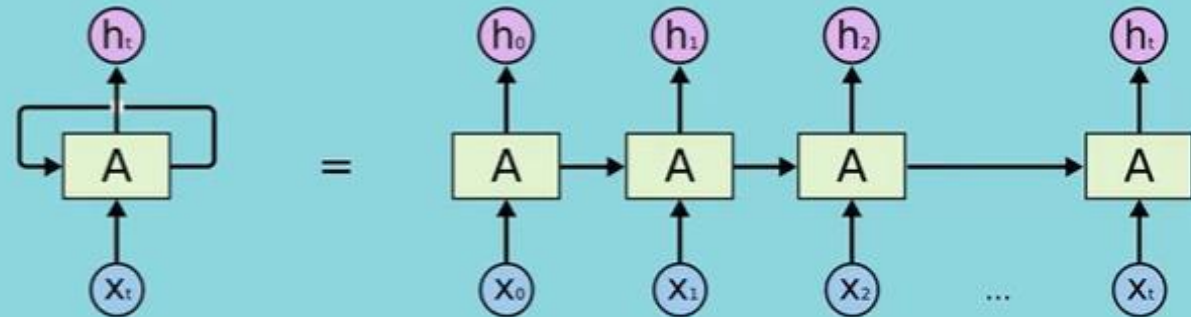
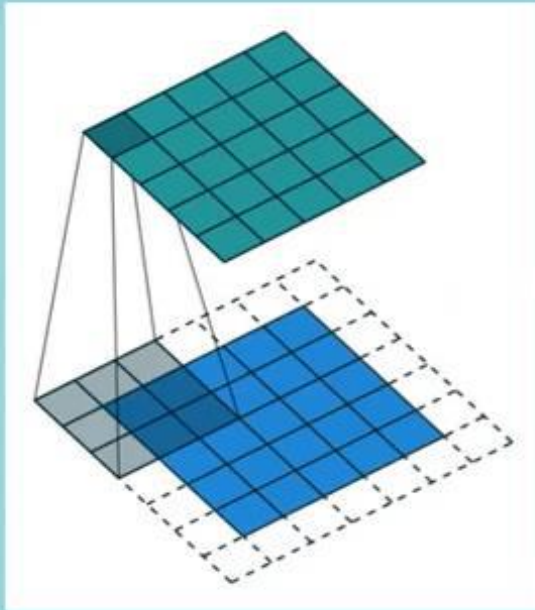
$$a_{ij} = \text{softmax}(\text{score}(x_i, x_j))$$

$$a_i = \sum_{j \leq i} a_{ij} v_j$$

$$\mathbf{Q} = \mathbf{XW}^Q; \mathbf{K} = \mathbf{XW}^K; \mathbf{V} = \mathbf{XW}^V \quad \mathbf{A} = \text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

Blank Slide

CNN, RNN and Transformers



Pro: Compute in Parallel

Con: Finite Horizon

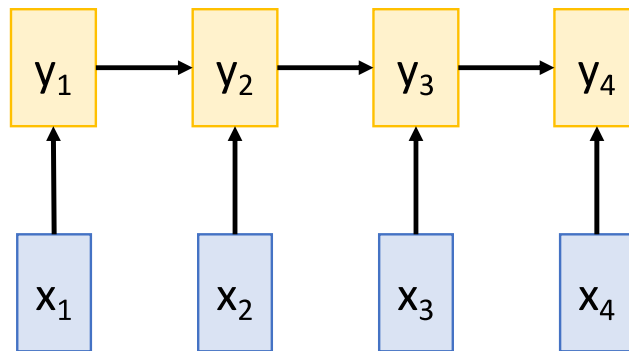
Pro: Infinite Horizon

Con: Compute Serially only

Attention

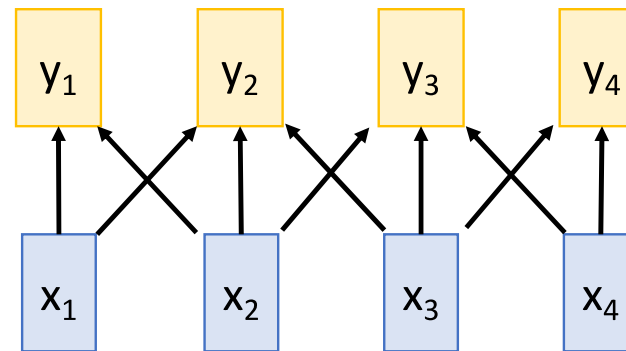
Conceptual Comparison

Recurrent Neural Network



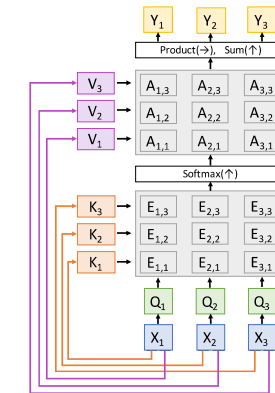
- Works on Ordered Sequences
- Long range dependence (in the past)
- Not parallelizable

1D Convolution



- Works on Multidimensional grids
- Short visibility; Stacking helps
- Highly parallelizable

Self-Attention



- Works on Sets and Sequences
- Long range dependence
- Parallelizable
- Memory Intensive

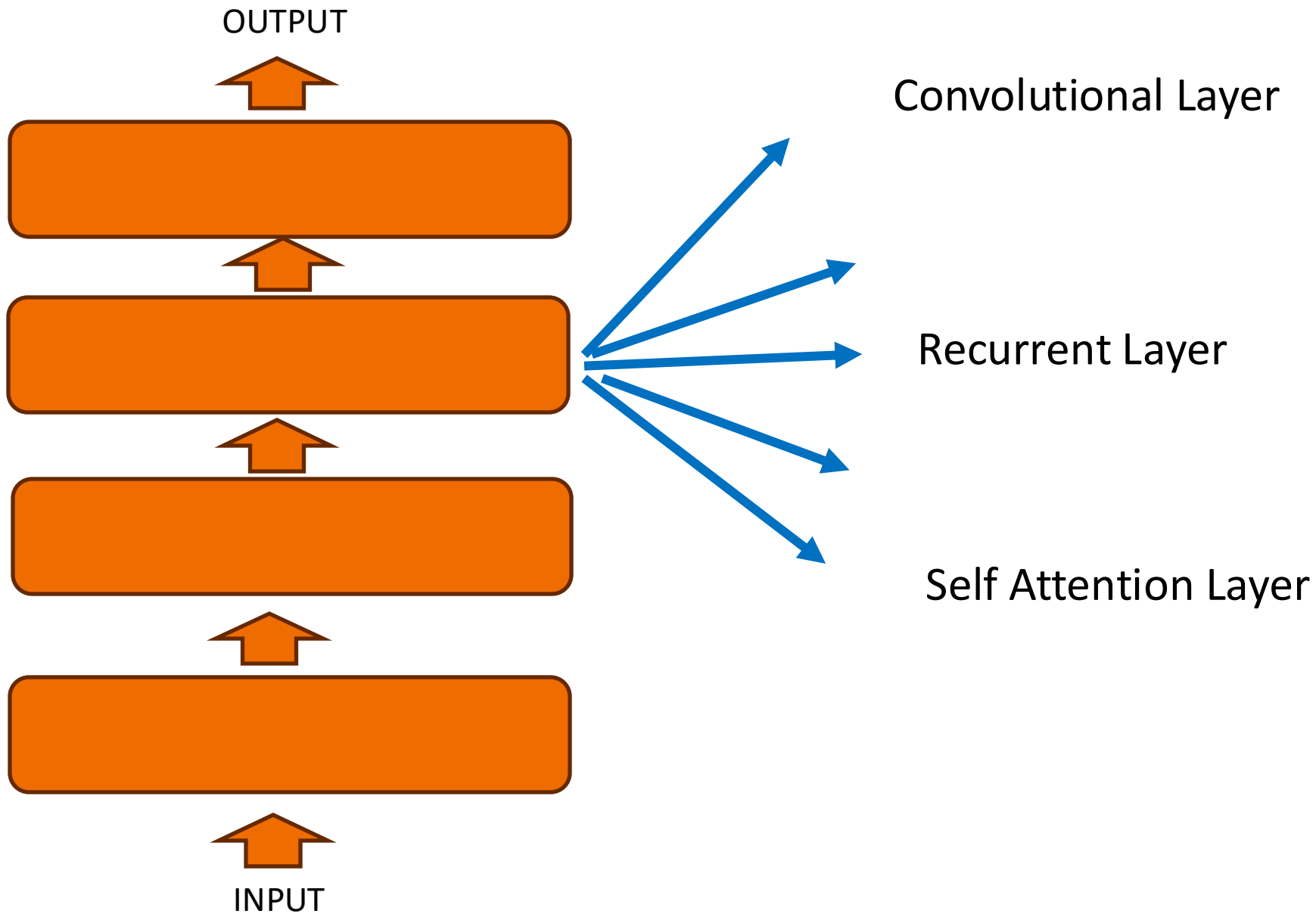
Self Attention Layer

Input: A set/sequence of vectors

Output: A set/sequence of vectors

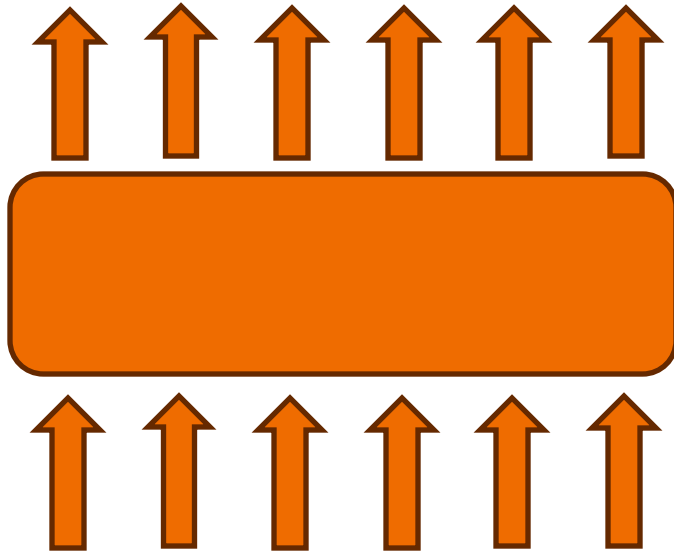
A naïve comparison; Primary for Comparison; Need not be accurate in numbers.

DL and Different Types of Layers



MLP: Fully connected layer

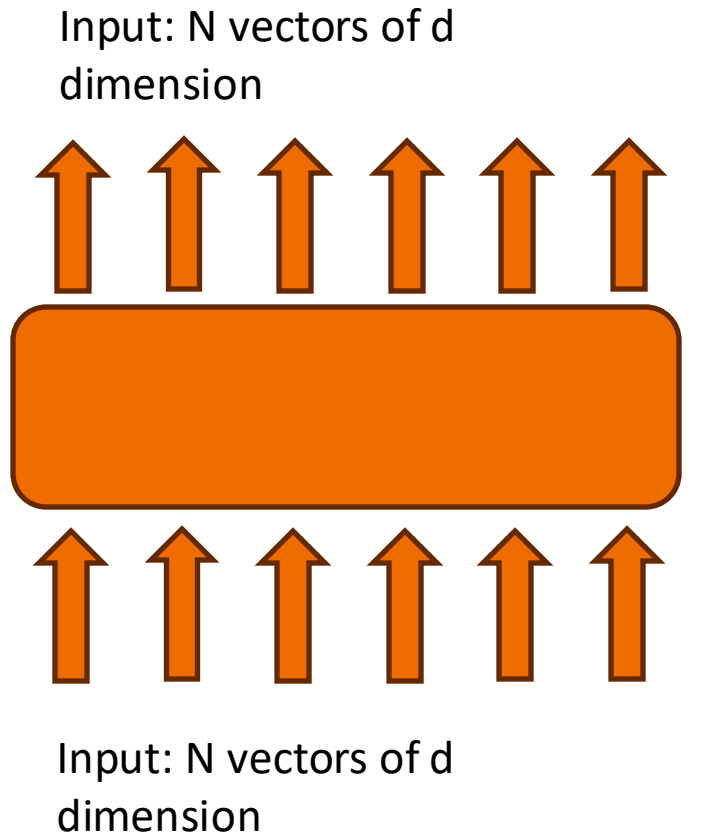
Input: N vectors of d
dimension



Input: N vectors of d
dimension

- Let $N=1000$ and $d = 300$
- Number of parameters to learn
 - $(1000*300) \times (1000*300)$
 - $=9 \times 10^{10}$
- Every element in the output is connected, related to every element in the input
- Number of parameters is high

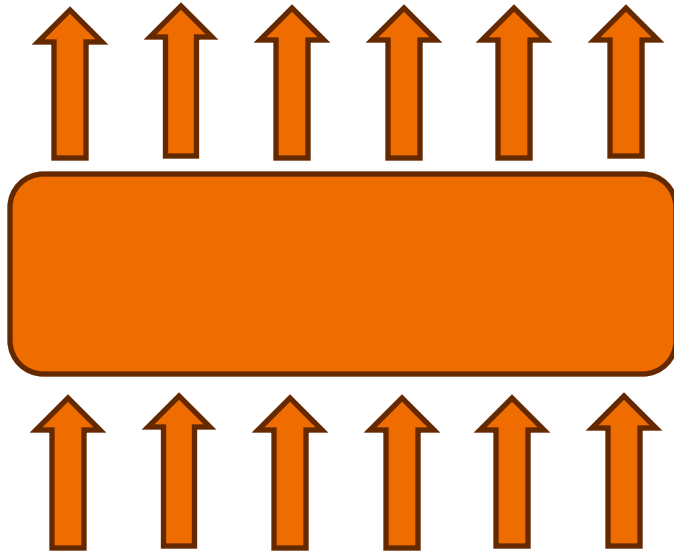
CNN: Convolutional Layer



- Every vector in the output is connected, related to a small set of neighboring element in the input (say 3X3)
 - The relationship in the locality remains same every where (weights are shared)
 - Boundary exceptions are taken care of (padding)
-
- Let $N=1000$ (say sqr) and $d = 300$
 - Number of parameters to learn
 - $300 \text{ (channels)} \times 3 \times 3 \times 300$
 - 27×10^4

RNN: Recurrent Seq2Seq Layer

Input: N vectors of d
dimension

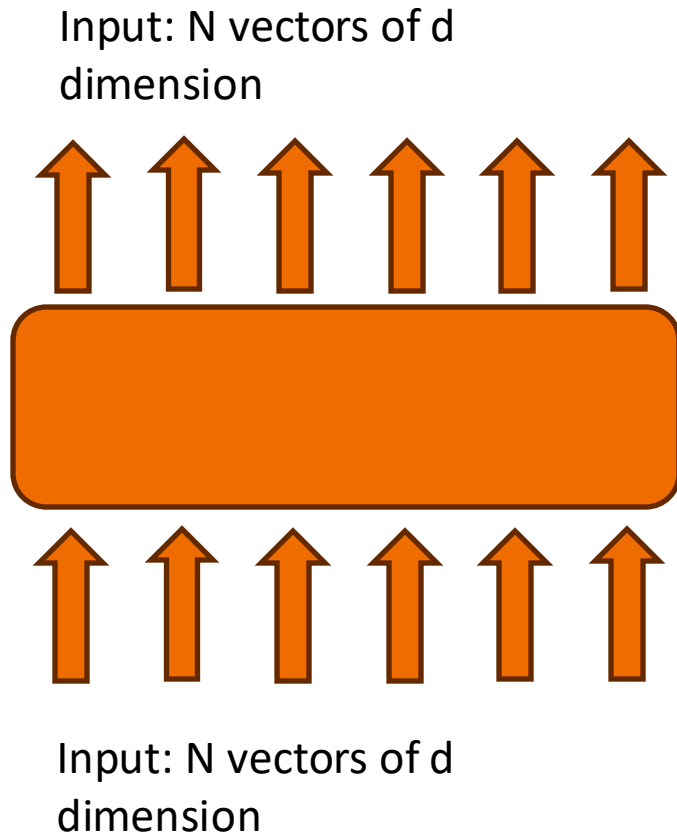


Input: N vectors of d
dimension

- Every vector in the output is related to the respective input (same time instance) vector and some history of the past
- Sequence of 1000 vectors
 - Let $N=1000$ and $d = 300$
 - (and $h = 100$)
 - Number of parameters to learn
 - 300×100 (U)
 - 100×100 (W)
 - 100×300 (V)
 - $= 7 \times 10^4$

Transformer: Self Attention layer

- Every vector in the output is related to every vector in the input in a lower dimension



$$q_i = x_i W^Q; k_i = x_i W^K; v_i = x_i W^V$$

$$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

$$a_{ij} = \text{softmax}(\text{score}(x_i, x_j))$$

$$a_i = \sum_{j \leq i} a_{ij} v_j$$

- Let $N=1000$ and $d = 300$
- Project to lower dimension; $h = 100$
 - $300 \times 100 (W^Q)$
 - $300 \times 100 (W^K)$
 - $300 \times 100 (W^V)$
 - $100 \times 300 (W^O)$
 - $= 12 \times 10^4$

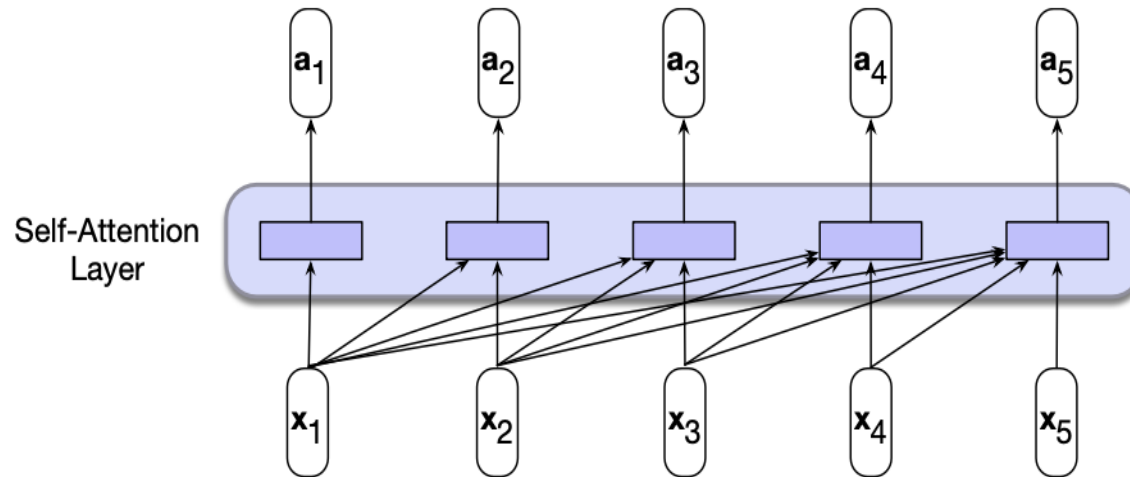
Blank Slide

Blank Slide

Masked and Multi Head Attention

Self Attention: Causal/masked

Masking out the future



Unlike RNNs, computations at each step are independent of all other time steps and can be done parallelly.

QK^T

N

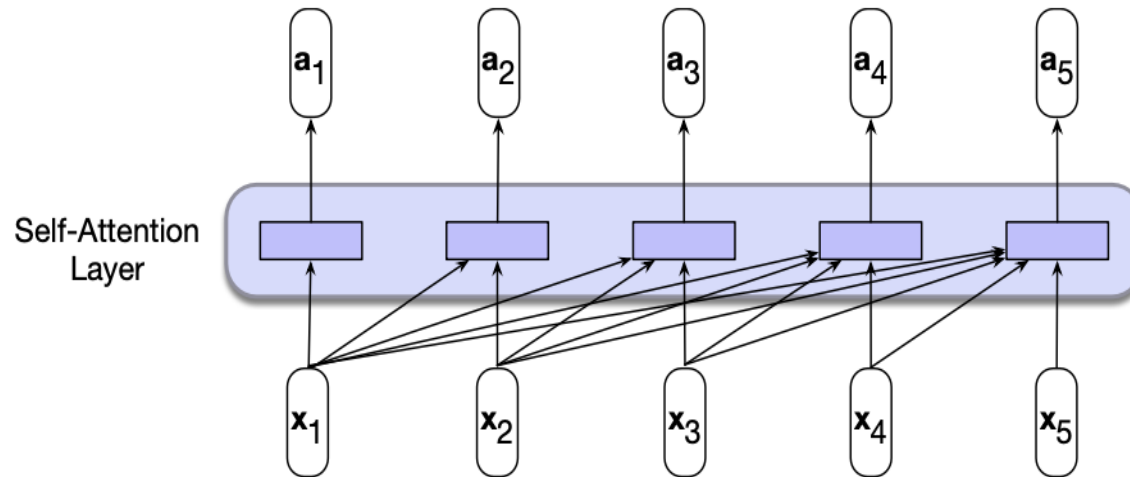
$q1 \cdot k1$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
$q2 \cdot k1$	$q2 \cdot k2$	$-\infty$	$-\infty$	$-\infty$
$q3 \cdot k1$	$q3 \cdot k2$	$q3 \cdot k3$	$-\infty$	$-\infty$
$q4 \cdot k1$	$q4 \cdot k2$	$q4 \cdot k3$	$q4 \cdot k4$	$-\infty$
$q5 \cdot k1$	$q5 \cdot k2$	$q5 \cdot k3$	$q5 \cdot k4$	$q5 \cdot k5$

N

What is the typical value of N?
4096 tokens

Self Attention: Causal/masked

Masking out the future



Unlike RNNs, computations at each step are independent of all other time steps and can be done parallelly.

What is the typical value of N?
4096 tokens

QK^T

N

q1•k1	−∞	−∞	−∞	−∞
q2•k1	q2•k2	−∞	−∞	−∞
q3•k1	q3•k2	q3•k3	−∞	−∞
q4•k1	q4•k2	q4•k3	q4•k4	−∞
q5•k1	q5•k2	q5•k3	q5•k4	q5•k5

N

$$attention(Q, K, V) = softmax\left(\frac{Q^T K}{\sqrt{d_k}}\right) V$$

$$maskedAttention(Q, K, V) = softmax\left(\frac{Q^T K + M}{\sqrt{d_k}}\right) V$$

where M is a mask matrix of 0's and $-\infty$'s

Masked Attention

Don't let vectors "look ahead" in the sequence

Inputs:

Input vectors: X (Shape: $N_X \times D_Q$)

Key matrix: W_K (Shape: $D_X \times D_Q$)

Value matrix: W_V (Shape: $D_X \times D_V$)

Query matrix: W_Q (Shape: $D_Q \times D_Q$)

Computation:

Query Vectors $Q = XW_Q$

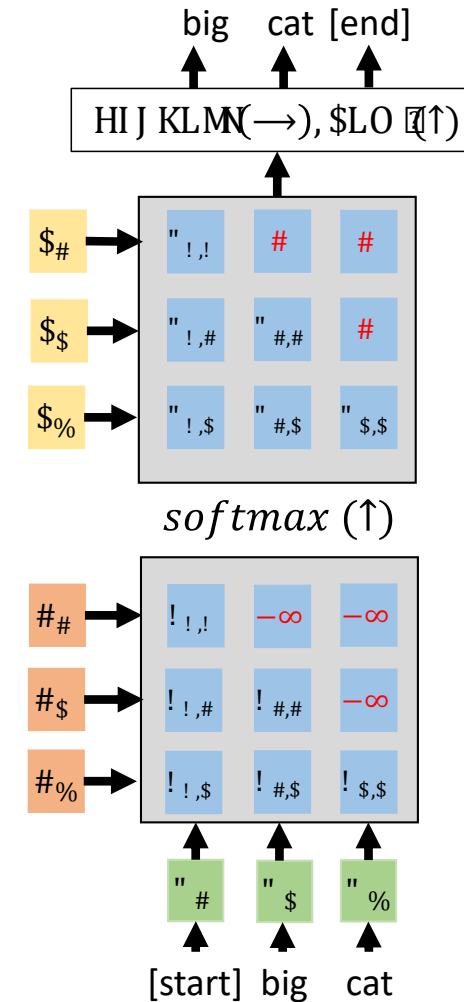
Key vectors: $K = XW_K$ (Shape: $N_n \times D_k$)

Value Vectors: $V = XW_V$ (Shape: $N_n \times D_v$)

Similarities: $E = \frac{Q \cdot K^T}{\sqrt{D_k}}$ (Shape: $N_n \times N_n$) $E_{\%} = (Q_{\%} \cdot K^T) / \sqrt{D_k}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (Shape: $N_n \times N_n$)

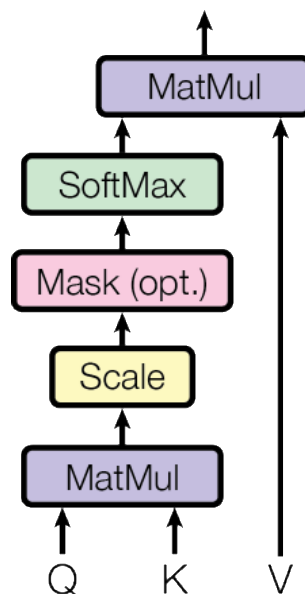
Output vectors: $Y = AV$ (Shape: $N_n \times D_v$) $Y_{\%} = \sum A_{\%} V$



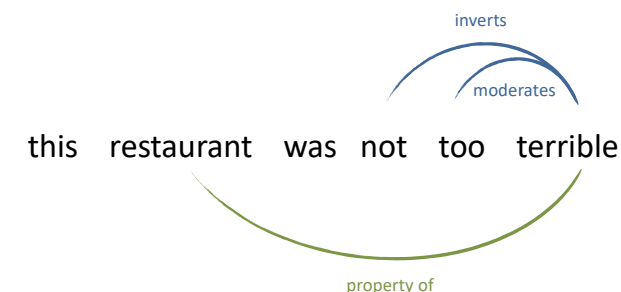
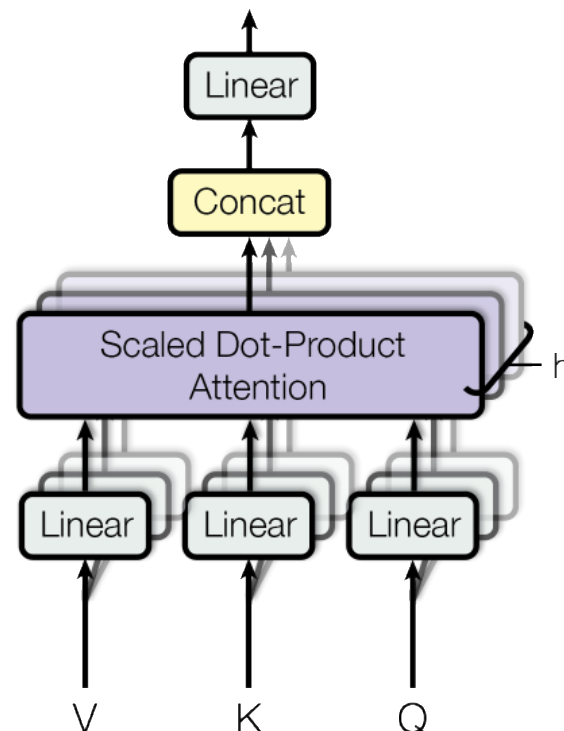
Multi Head Attention: Why?

- Why are we limited to one query, key and value?
- Multiple Feature Maps (like Convolution Layers)

Scaled Dot-Product Attention



Multi-Head Attention



Masked Multi-Head Attention

- Masked multi-head attention: multi-head where some values are masked (i.e., probabilities of masked values are nullified to prevent them from being selected).
- When decoding, an output value should only depend on previous outputs (not future outputs). Hence we mask future outputs.

$$attention(Q, K, V) = softmax \left(\frac{Q^T K}{\sqrt{d_k}} \right) V$$

$$maskedAttention(Q, K, V) = softmax \left(\frac{Q^T K + M}{\sqrt{d_k}} \right) V$$

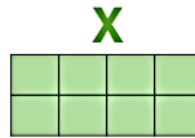
where M is a mask matrix of 0's and $-\infty$'s

Blank Slide

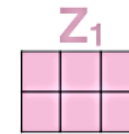
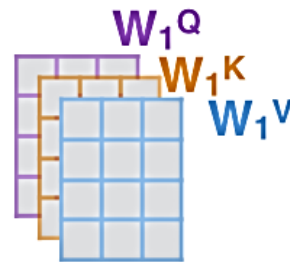
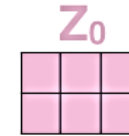
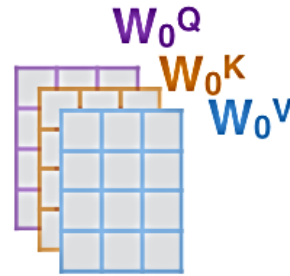
Multi-Head Attention

- 1) This is our input sentence*
- 2) We embed each word*
- 3) Split into 8 heads. We multiply X or R with weight matrices
- 4) Calculate attention using the resulting $Q/K/V$ matrices
- 5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

Thinking
Machines



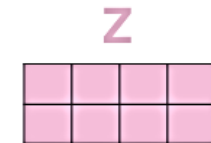
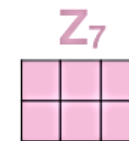
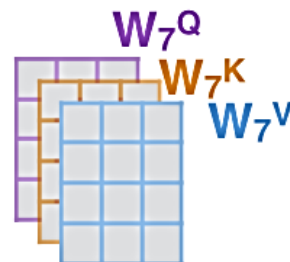
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



...

...

...



Blank Slide

Summary Computation

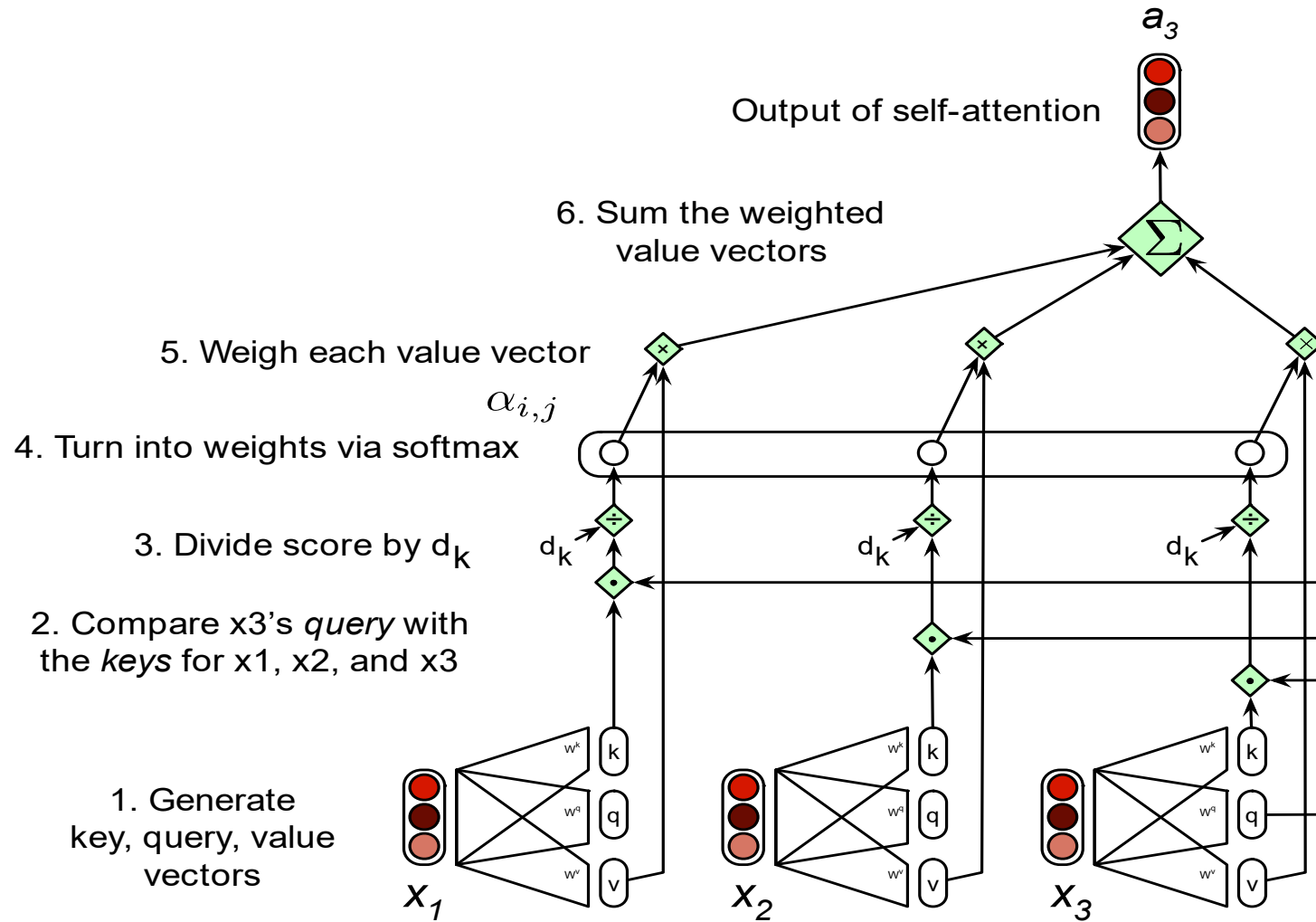
$$q_i = x_i W^Q; k_i = x_i W^K; v_i = x_i W^V$$

$$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{d_k}$$

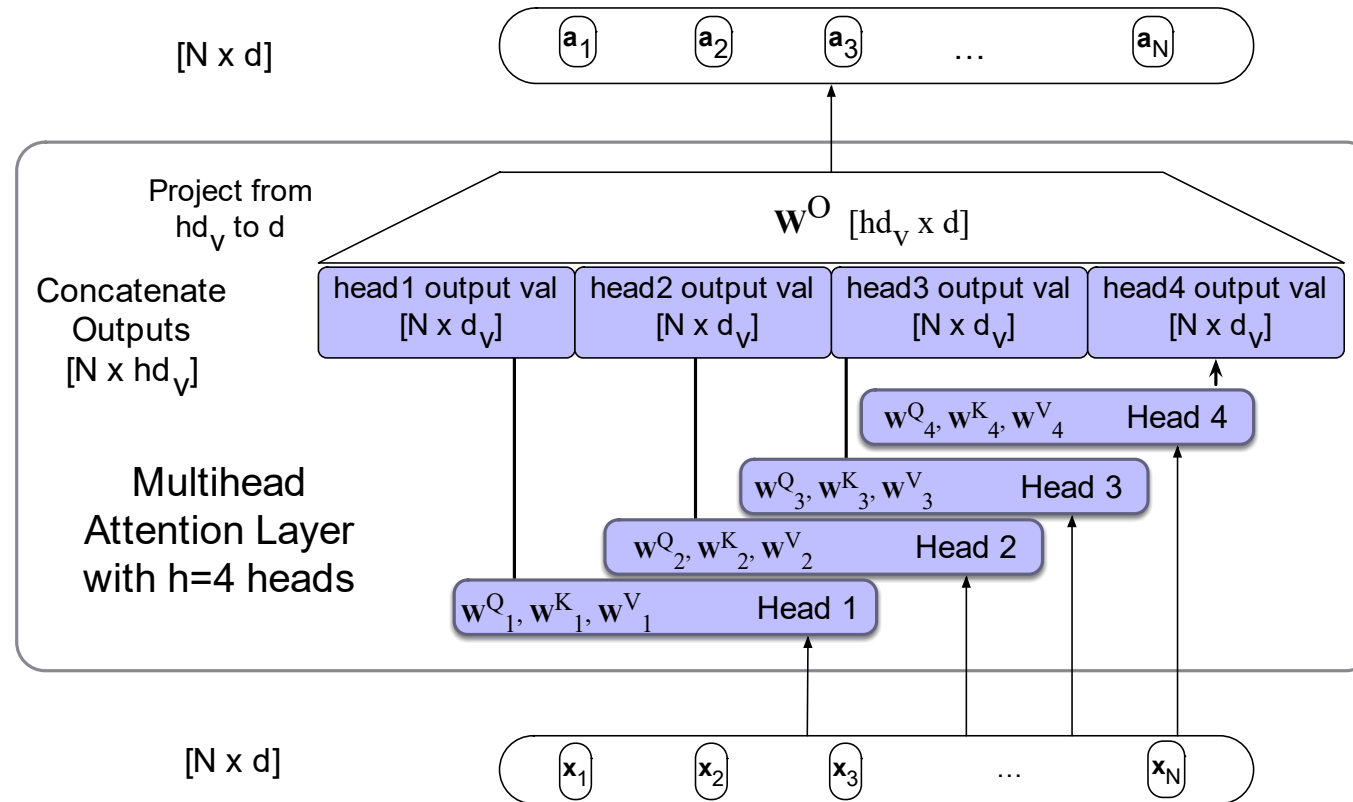
$$a_{ij} = \text{softmax}(\text{score}(x_i, x_j))$$

$$a_i = \sum_{j \leq i} a_{ij} v_j$$

Summary



Summary Multi Head Attention



$$\mathbf{Q} = \mathbf{XW}_i^Q ; \mathbf{K} = \mathbf{XW}_i^K ; \mathbf{V} = \mathbf{XW}_i^V$$

$$\text{head}_i = \text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$$

$$\mathbf{A} = \text{MultiHeadAttention}(\mathbf{X}) = (\text{head}_1 \oplus \text{head}_2 \dots \oplus \text{head}_h) \mathbf{W}^O$$

Thanks!!

Questions?